

The Facility Location Problem

Yuhao “Ben” Pan

1 The Facility Location Problem

Given:

- A set of facilities, F , and a set of clients, C .
- Each connection between a facility i and a client j has an associated cost, c_{ij} .
- Each facility i has a setup cost of f_i .
- Defined on a metric space, i.e., the costs follow the triangle inequality (we will get to that in section 5.2).

Goals:

- Choose a subset of facilities and open them.
- Assign/connect each client to **exactly one** facility.
- Minimize the overall cost (facility + connection).

Consider the following scenario: Imagine you are a shipping company like FedEx or UPS. You are looking to expand into New York State and build warehouses (facilities) to store goods. You have some major cities to serve, like New York, Buffalo, Syracuse, Albany, etc. You also have some potential locations in mind. How can you choose some of these locations to open warehouses while minimizing the total distance between the cities and their assigned warehouses so that goods can be delivered faster? This is a real life example of what is called the facility location problem.

2 Introduction to Linear Programming

Before we get into the details of the problem, we will introduce the concept of linear programming. Suppose we are given a maximization problem to be formulated in linear programming in the following way:

Maximize

$$c_1x_1 + c_2x_2 + \cdots + c_nx_n,$$

subject to the following m constraints:

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq b_1$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \leq b_2$$

$$\vdots$$

$$a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \leq b_m$$

$$x_1, x_2, \cdots, x_n \geq 0.$$

We can think of c_i as “cost coefficients” and x_i as decision variables. We will also define the **objective function** as the expression we are trying to maximize, namely $c_1x_1 + c_2x_2 + \cdots + c_nx_n$.

2.1 Dual LP

Suppose we solve a linear program, like the one above, and we want an alternate method to verify that our solution is correct. How can we do that? The answer lies in the **dual** of a linear program.

The dual of a linear program is simply another equivalent linear program based off of the original LP, or the **primal** LP. By solving the dual LP, we obtain an optimal solution that should be the same as the optimal solution given by solving the primal LP. In other words, solving the dual LP verifies the solution of the corresponding primal LP.

2.2 Derivation of Dual from Primal

Let us use the standard primal LP above.

First, we multiply each constraint by a constant y_i :

$$\begin{aligned} y_1a_{11}x_1 + y_1a_{12}x_2 + \cdots + y_1a_{1n}x_n &\leq y_1b_1 \\ y_2a_{21}x_1 + y_2a_{22}x_2 + \cdots + y_2a_{2n}x_n &\leq y_2b_2 \\ &\vdots \\ y_ma_{m1}x_1 + y_ma_{m2}x_2 + \cdots + y_ma_{mn}x_n &\leq y_mb_m. \end{aligned}$$

Then, we add all constraints and combine like terms:

$$\begin{aligned} (y_1a_{11} + y_2a_{21} + \cdots + y_ma_{m1})x_1 + \\ (y_1a_{12} + y_2a_{22} + \cdots + y_ma_{m2})x_2 + \\ \vdots \\ (y_1a_{1n} + y_2a_{2n} + \cdots + y_ma_{mn})x_n &\leq y_1b_1 + y_2b_2 + \cdots + y_mb_m. \end{aligned}$$

Since we have an upper bound of some linear function of x ($y_1b_1 + y_2b_2 + \cdots + y_mb_m$), the logical way of formulating an alternative linear program is to minimize the upper bound so that it is as close as possible to the maximum of the objective function.

Therefore, in the dual LP, we will minimize $y_1b_1 + y_2b_2 + \cdots + y_mb_m$.

Just like the primal LP, the dual has to satisfy a set of constraints. Since we want the upper bound to satisfy the maximum of the objective function, we want the coefficients of the new constraints to be greater than the cost coefficients (otherwise our upper bound is invalid):

$$\begin{aligned} y_1a_{11} + y_2a_{21} + \cdots + y_ma_{m1} &\geq c_1 \\ y_1a_{12} + y_2a_{22} + \cdots + y_ma_{m2} &\geq c_2 \\ &\vdots \\ y_1a_{1n} + y_2a_{2n} + \cdots + y_ma_{mn} &\geq c_n. \end{aligned}$$

This completes our dual LP.

2.3 Matrix Form of Primal and Dual

We can write the LPs above using matrices:

$$\text{Let } C = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}, X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}, Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Primal:

Maximize $C^T X$, subject to the constraints:

$$\begin{aligned} AX &\leq B \\ X &\geq 0. \end{aligned}$$

Dual:

Minimize $B^T Y$, subject to the constraints:

$$\begin{aligned} A^T Y &\geq C \\ Y &\geq 0. \end{aligned}$$

2.4 How to Solve Linear Programs: The Simplex Method

There are many algorithms to solve linear programs, of which the most well-known is the Simplex Method. We won't go into the details of the algorithm, but we will explain the intuition behind it. Linear programs are essentially a system of linear inequalities, and it is a well-known result in mathematics that a system of linear inequalities defines a convex polyhedron. Intuitively, you would expect the extremum of the objective function to occur at a vertex of the polyhedron, so the Simplex Method first chooses a vertex of the polyhedron.

In a maximization problem, we will proceed as follows. During each iteration of the algorithm, we scan the neighbors of the current vertex to determine which vertex, when moved to, will increase the value of the objective function at the fastest rate. Then we move to that vertex and repeat. The algorithm terminates when, during the scanning step, moving to any neighbor will result in a decrease of the value of the objective function, which means the current vertex is the maximum solution.

The Simplex Method will only work for maximization problems. However, for minimization problems, we can simply apply the Simplex Method to their duals.

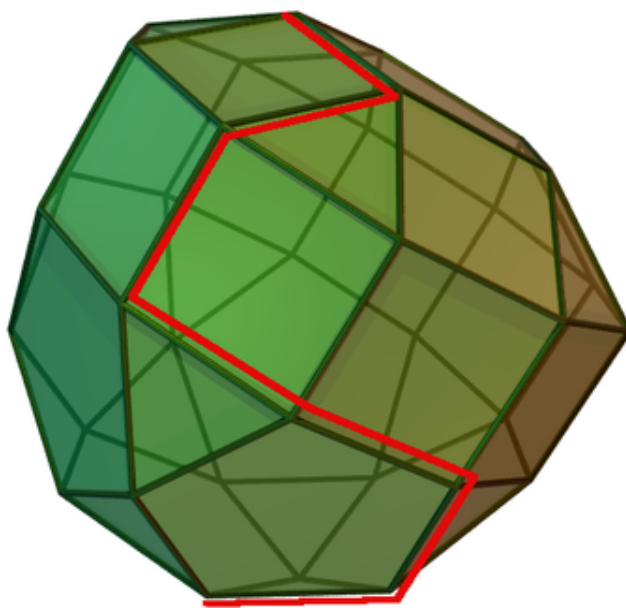


Figure 1: The path produced by our “scan”

3 Theorems of Primal/Dual

To better understand the relationship between the primal and dual, we will introduce some theorems that will also be useful later.

3.1 The Dual of the Dual is the Primal

To form the dual of the dual, we simply perform the same process by which we convert the primal to the dual, i.e., flipping of signs and matrix variables. Flipping signs and matrix variables twice would result in the original signs and variables, so the dual of the dual is simply the primal.

3.2 Strong Duality Theorem

3.2.1 Theorem

At optimality, the primal objective function value equals the dual objective function value.

3.3 Weak Duality Theorem

3.3.1 Theorem

Let $(x_1, x_2, \dots, x_n)$ and $(y_1, y_2, \dots, y_m)$ be feasible solutions to the primal (maximization) and dual (minimization), respectively. Then,

$$\sum_{j=1}^n c_j x_j \leq \sum_{i=1}^m b_i y_i.$$

In other words, any primal feasible solution is less than or equal to any dual feasible solution.

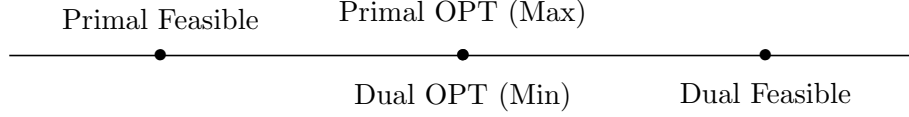


Figure 2: The relationships between feasible and optimum solutions in a maximization problem

3.3.2 Proof

$$\begin{aligned}
 \sum_{j=1}^n c_j x_j &\leq \sum_{j=1}^n \sum_{i=1}^m (a_{ij} y_i) x_j \\
 &= \sum_{i=1}^m y_i \sum_{j=1}^n a_{ij} x_j \\
 &\leq \sum_{i=1}^m y_i b_i.
 \end{aligned}$$

3.4 Complementary Slackness Condition

3.4.1 Theorem

Primal version: Either $x_j^* = 0$ or $\sum_{i=1}^m a_{ij} y_i = c_j$.

Dual version: Either $y_i^* = 0$ or $\sum_{j=1}^n a_{ij} x_j = b_i$.

3.4.2 Remark

Essentially, CSC describes the “slackness” of the constraints between a primal or dual LP. A constraint is “slack” if it is an inequality and “tight” if it is an equality. A variable is “slack” if it is positive and “tight” if it is 0. CSC claims that a primal variable and its corresponding constraint in the dual (or vice versa) cannot have slack at the same time. Otherwise, the solution to the LP is not optimal (“slack”) and needs to be “tightened.”

4 The Facility Location Problem, Revisited with LP

4.1 LP-formulation

Let $x_{ij} = 1$ if facility i is connected to client j , 0 otherwise.

Let $y_i = 1$ if facility i is open, 0 otherwise.

The facility location problem can then be written as a linear program:

Minimize

$$\sum_{i \in F} f_i y_i + \sum_{i \in F, j \in C} c_{ij} x_{ij},$$

subject to the following constraints:

$$\begin{aligned} x_{ij} &\leq y_i, \forall i, j \\ \sum_{i \in F} x_{ij} &\geq 1, \forall j \\ x_{ij} &\in \{0, 1\} \subseteq \mathbb{Z} \\ y_i &\in \{0, 1\} \subseteq \mathbb{Z}. \end{aligned}$$

Note that this linear program is a bit different from the standard linear programs. There are **integrality** constraints that must be satisfied. This is known as an **integer linear program**. In general, integer linear programs are known to be “unsolvable” in a reasonable amount of time.

4.1.1 Remark for readers who are familiar with time complexity and P vs NP

There are many algorithms that can solve a standard linear program within polynomial time. (The number of “steps” in the algorithm can be upper-bounded by a polynomial function in which the exponents are constants. In simpler terms, polynomial time = reasonable time.) However, there is no algorithm that can solve an **integer linear program** within polynomial time because you can reduce the vertex cover problem to an ILP, but the vertex cover problem is NP-hard, so integer linear programs are NP-hard as well.

Therefore, we must convert the integer linear program above to a standard linear program without integrality constraints, a process known as **LP-relaxation**. Removing the integrality constraints, we have the following linear program:

Minimize

$$\sum_{i \in F} f_i y_i + \sum_{i \in F, j \in C} c_{ij} x_{ij},$$

subject to the following constraints:

$$y_i \geq x_{ij} \forall i, j \tag{Primal 1}$$

$$\sum_{i \in F} x_{ij} \geq 1, \forall j \tag{Primal 2}$$

$$x_{ij} \geq 0.$$

However, LP-relaxation will sacrifice the accuracy of the resulting optimum solution.

4.1.2 Theorem

Given an ILP, the LP after relaxation will produce an optimum solution that is less than or equal to the optimum solution of the original ILP.

4.1.3 Remark

Note that the optimum solution of the ILP is also a feasible solution of the relaxed LP, since it satisfies all constraints of the relaxed LP. (The converse is not necessarily true.) In a minimization problem, the optimum solution is less than or equal to all feasible solutions.

4.2 Dual LP

In section 2.2, we discussed how to derive the dual from the primal. We can apply the same procedure to our primal LP above. First, multiply each constraint by some variables α and β which will become our dual variables, corresponding to x , y respectively.

$$y_i(\beta_{ij}) \geq (\beta_{ij})x_{ij} \quad \forall i, j \quad (1)$$

$$(\alpha_j) \sum_{i \in F} x_{ij} \geq \alpha_j \quad \forall j. \quad (2)$$

Comparing the coefficients of y_i , we have:

$$\begin{aligned} \sum_j (\beta_{ij}) y_i &\leq f_i y_i \quad \forall i \\ \sum_j (\beta_{ij}) &\leq f_i \quad \forall i. \end{aligned}$$

Now we compare the coefficients of x_{ij} . First we can rearrange (1) and (2):

$$-(\beta_{ij})(x_{ij}) \geq -(\beta_{ij})(y_i), \text{ and}$$

$$(\alpha_j) \sum_{i \in F} x_{ij} \geq \alpha_j.$$

Next, combine the left-hand side of both inequalities and compare their coefficients of x_{ij} with c_{ij} :

$$\begin{aligned} \alpha_j \left(\sum_{i \in F} x_{ij} \right) - \beta_{ij} x_{ij} &\leq c_{ij} x_{ij}, \quad \forall i, j \\ \alpha_j x_{ij} - \beta_{ij} x_{ij} &\leq c_{ij} x_{ij} \\ \alpha_j - \beta_{ij} &\leq c_{ij} \end{aligned}$$

Thus, the dual is as follows:

Maximize

$$\sum_j \alpha_j,$$

subject to the following constraints:

$$\sum_j \beta_{ij} \leq f_i \quad \forall i \quad (\text{Dual 1})$$

$$\begin{aligned} \alpha_j - \beta_{ij} &\leq c_{ij} \quad \forall i, j \\ \alpha_j, \beta_{ij} &\geq 0 \quad \forall i, j. \end{aligned} \quad (\text{Dual 2})$$

5 Introduction to Approximation Algorithms

Approximation algorithms are a class of algorithms that find feasible solutions to “unsolvable” (NP-hard) optimization problems in a reasonable amount of time. (The technical term is **polynomial time**, which means the running time function can be upper bounded using the Big-O notation by a polynomial expression.)

To measure the effectiveness of an approximation algorithm, we will use a metric called **performance guarantee**. For example, in a minimization problem, an approximation algorithm that produces a feasible solution at most 4 times the optimum solution has an approximation guarantee of 4. As a shorthand, we will call this algorithm a **4-approximation algorithm**.

5.1 A 4-Approximation Algorithm for the Facility Location Problem

Now we have both the primal and dual. We will define some important terms:

- Let (x^*, y^*) and (α^*, β^*) be the optimum solution to the primal and the dual, respectively.
- Let Primal OPT, Dual OPT be the value of the objective function of the relaxed LP evaluated by (x^*, y^*) and (α^*, β^*) , respectively.
- Note that by the Strong Duality Theorem, Primal OPT = Dual OPT.
- Let OPT be the optimum solution to the ILP. In other words, OPT is the desired (and correct) solution to the original problem, without LP-relaxation.
- Note that by Theorem 4.1.2, Primal OPT = Dual OPT $\leq$ OPT.

5.1.1 Definition

$N_j = \{i \in F | x_{ij}^* > 0\}$ represents a neighborhood of client j .

The antecedent $x_{ij}^* > 0$ means that client j gets fractional service from facility i in our optimum solution.

5.1.2 Lemma

If $x_{ij}^* > 0$, then $c_{ij} \leq \alpha_j^*$.

5.1.3 Proof

Primal CSC states that either $x_{ij}^* = 0$ or $\alpha_j^* - \beta_{ij}^* = c_{ij}$.

Since $x_{ij}^* > 0$, $\alpha_j^* - \beta_{ij}^* = c_{ij}$.

Since $\beta_{ij}^* \geq 0$, $c_{ij} \leq \alpha_j^*$.

5.1.4 The 4-Approximation Algorithm

Our algorithm can be broken down into the following steps:

Step 1: Solve the primal and dual LP optimally to yield $(x^*, y^*), (\alpha^*, \beta^*)$.

Step 2: Pick a client with the smallest α^* value.

Step 3: Choose the cheapest facility to open in its neighborhood and assign the client to the facility.

Step 4: Remove all clients whose neighborhoods overlap with the client chosen in step 3.

Step 5: Repeat steps 2 through 4 until there are no more clients left.

Step 6: All clients that were removed in step 4 are assigned to the “nearest” open facility.

5.2 Analysis: Bounding the Connection Costs $\sum c_{ij}$

The most interesting step in this algorithm is the removal of all clients whose neighborhoods overlap with the neighborhood of the chosen client.

In other words, for the client j with the smallest α^* , we remove all j' if $N_j \cap N_{j'} \neq \emptyset$.

We want to be able to bound the connection cost between the removed client j' and some facility i opened by client j , using an intermediate facility i' “shared” by both j and j' .

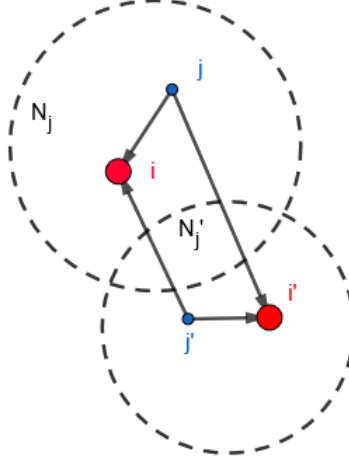


Figure 3: Overlapping neighborhoods

More formally, let j' be a client removed in step 4 of our algorithm.

Let j be the nearest open facility of j' . N_j must overlap with $N_{j'}$ because otherwise $N_{j'}$ will have no neighborhood “shared” with any open facility, but that would also mean that j' is not removed, which is a contradiction.

Let i be the facility opened when client j was chosen in step 2, and let i' be a facility such that $i' \in N_j \cap N_{j'}$.

Then we can write:

$$\begin{aligned}
 c_{ij'} &\leq c_{ij} + c_{i'j} + c_{i'j'} \text{ by the triangle inequality} \\
 &\leq \alpha_j^* + \alpha_j^* + \alpha_{j'}^* \text{ by Lemma 5.1.2} \\
 &\leq \alpha_{j'}^* + \alpha_{j'}^* + \alpha_{j'}^* \text{ by step 2 of our algorithm} \\
 &\leq 3\alpha_{j'}^*.
 \end{aligned}$$

5.2.1 Theorem

The connection cost of removed clients can be upper bounded by $3\alpha_j^* = 3\text{Dual OPT}$.

Let's consider the connection cost of clients that were not removed, i.e., vertices that opened some facility in step 3. This is fairly straightforward since such a client is assigned to a facility that is in its neighborhood.

Therefore, we can apply Lemma 5.1.2 to upper bound the connection cost by α_j^* .

5.2.2 Theorem

The connection cost of non-removed clients can be upper bounded by $\alpha_j^* = \text{Dual OPT}$.

Combining Theorems 5.2.1 and 5.2.2, we can conclude that:

5.2.3 Theorem

The total connection cost can be upper bounded by 3 Dual OPT.

5.3 Analysis: Bounding the Facility Costs

5.3.1 Lemma

$$\sum_{i \in N_j} y_i^* \geq 1.$$

5.3.2 Proof

From Primal 2, we have

$$\sum_{i \in F} x_{ij}^* \geq 1.$$

We know that $N_j = \{i | x_{ij}^* > 0\}$, so $\forall i \in F \setminus N_j$,

$$x_{ij}^* = 0.$$

Thus,

$$\sum_{i \in N_j} x_{ij}^* = \sum_{i \in F} x_{ij}^* \geq 1.$$

From Primal 1, $y_i \geq x_{ij} \quad \forall i, j$, so

$$\sum_{i \in N_j} y_i^* \geq \sum_{i \in N_j} x_{ij}^* \geq 1.$$

Let $i_0(j)$ be the facility with the minimum opening cost in N_j , and let $f_{i_0(j)} = \min f_i, \forall i \in N_j$. Then, we have:

5.3.3 Lemma

$$f_{i_0(j)} \leq \sum_{i \in N_j} f_i y_i^*.$$

5.3.4 Proof

By Lemma 5.3.1,

$$\sum_{i \in N_j} y_i^* \geq 1.$$

Thus,

$$\begin{aligned} f_{i_0(j)} &\leq f_{i_0(j)} \left(\sum_{i \in N_j} y_i^* \right) \\ &= \sum_{i \in N_j} f_{i_0(j)} y_i^* \\ &\leq \sum_{i \in N_j} f_i y_i^* \text{ since } f_{i_0(j)} \leq f_i, \forall i. \end{aligned}$$

5.3.5 Bounding the Facility Cost

Let D be the set of open facilities. Now we can bound the total facility cost:

$$\begin{aligned} \text{Total facility cost} &= \sum_{j \in D} f_{i_0(j)} \\ &\leq \sum_{j \in D} \sum_{i \in N_j} f_i y_i^* \text{ by Lemma 5.3.1} \\ &= \sum_{k \in F} f_k y_k^* \\ &\leq \sum_{k \in F} f_k y_k^* + \sum_{i \in F, j \in C} c_{ij} x_{ij} \\ &= \text{Primal OPT}. \end{aligned}$$

5.3.6 Theorem

The total facility cost can be upper bounded by Primal OPT; combining Theorems 5.3.3 and 5.2.3, we have:

$$\begin{aligned} \text{Total Cost} &= \text{Connection Cost} + \text{Facility Cost} \\ &\leq 3 \text{ Dual OPT} + \text{Primal OPT} \\ &\leq 4 \text{ Primal OPT} \\ &\leq 4 \text{ OPT by Theorem 4.1.2.} \end{aligned}$$

6 Conclusion

Our algorithm yields a solution that is guaranteed to be at most 4 times the optimum solution, making it a “4-approximation algorithm.” The gist of the algorithm is filtering (removing clients in step 4) and LP-rounding (opposite of LP-relaxation, represented by the assignment of removed clients to open facilities). The algorithm was created by Shmoys, Tardos, and Aardal in 1997. There are other algorithms developed recently that overperform the algorithm presented in this article, up to an approximation ratio of 1.52!

The facility location problem also has many variants. The variant presented in this article is called the “Uncapacitated Facility Location Problem,” where each facility has no restrictions on how many clients it can serve. There is a capacitated version in which each facility has a capacity and each client has a demand. The problem can be extended to the k -center problem and more generally, cluster analysis. The k -center problem is the same as the facility location problem with one caveat: you want to minimize the maximum distance, not the total distance, between facilities and clients. Both the facility location problem and the k -center problem belong to a set of problems known as cluster analysis, in which one is asked to group objects such that objects in the same group are the most similar. In the case of the facility location problem, the groups are simply the set of all clients connected to a facility which are as close (“similar”) to the facility as possible.

References

- Bumb, A. (2002). (thesis). Approximation Algorithms for Facility Location Problems.
December 16, 2021, from <https://ris.utwente.nl/ws/portalfiles/portal/6073555/t000001e.pdf>.
- Nagarajan, V. (2017, February). Metric Facility Location (LP Rounding).
IOE 691: Approximation Algorithms . Retrieved December 16, 2021, from
<http://viswa.engin.umich.edu/wp-content/uploads/sites/169/2016/12/10.pdf>.
- Trevisan, L. (2011, January). Lecture 6. CS261: Optimization. Retrieved December 11, 2021,
from <https://theory.stanford.edu/~trevisan/cs261/lecture06.pdf>.